

Resumen General para el Parcial — OO2 (Refactoring, Bad Smells, Patrones y Detección sobre AST)

Compilado de: [CodeSmells.md](#), [Practica_BadSmells.md](#), [ResolucionPractiaCursada.md](#), [Resolucion.md](#) (Deep into Refactoring), [Patrones.md](#) y [Repaso.md](#).

1. Definiciones importantes

Concepto	Definición
Refactoring	Transformación del código que mejora su estructura interna sin cambiar su comportamiento observable . Si cambia el comportamiento (ej. corregir un bug o cambiar una firma), ya no es refactoring (puede ir <i>acompañado</i> de refactorings, pero es un cambio de funcionalidad).
Code smell / Bad smell	Señal en el código de que existe un problema de diseño más profundo. No es un error : el programa funciona, pero será difícil de mantener, entender o modificar.
Patrón de diseño	Solución reutilizable y probada a un problema recurrente de diseño, en un contexto dado. Es una <i>plantilla</i> (descripción de roles y colaboraciones), no código copiable tal cual. Los clásicos son los 23 del GoF.
Anti-patrón	Solución tentadora y común que en la práctica empeora el diseño (ej. Dios Objeto, Spaghetti Code).
Deuda técnica	El "costo a futuro" que se acumula por dejar código sucio hoy; los smells son la moneda de esa deuda.
Comportamiento observable	Lo que el programa hace hacia afuera: resultados, salida, efectos, excepciones. El refactoring puede tocar <i>cómo</i> lo hace, nunca <i>qué</i> hace.
Cohesión	Qué tan relacionado está lo que hace una clase/método. Alta cohesión = hace <i>una</i> cosa bien.
Acoplamiento	Cuánto depende una clase de otra. Bajo acoplamiento = se puede cambiar una clase sin romper las demás.
DRY	<i>Don't Repeat Yourself</i> : la duplicación es la raíz de muchos smells (Duplicated Code, Shotgun Surgery).
Open/Closed Principle	Abierto para extensión, cerrado para modificación: agregar un tipo nuevo debería ser agregar una clase, no tocar un switch (clave para justificar <i>Replace Conditional with Polymorphism</i>).
AST (Árbol Sintáctico Abstracto)	Árbol que representa la estructura del código fuente (nodos = funciones, sentencias, expresiones; hojas = identificadores, literales). Es lo que se recorre para <i>detectar</i> smells automáticamente.

Concepto	Definición
ANTLR / Gramática	Herramienta y formalismo para definir un lenguaje (como el "pool" de la cátedra) y generar el parser que produce el AST que se analiza en ANTLR Lab.
Null Object	Objeto que implementa el protocolo completo de un tipo con respuestas neutras/inofensivas (cadena vacía, false , no-op, Integer.MIN_VALUE documentado) para eliminar chequeos null .
Primitiva vs Hook	En Template Method: la <i>primitiva</i> es un paso abstracto que la subclase debe implementar; el <i>hook</i> es un paso opcional con implementación por defecto en la clase abstracta.
Refactoring vs Patrón	El refactoring <i>lleva</i> el código hacia un patrón: <i>Form Template Method</i> → Template Method; <i>Replace Conditional with Polymorphism</i> → Strategy/State; <i>Introduce Null Object</i> → Null Object.

2. Bad Smells — glosario y corrección

En el parcial vale el nombre **en inglés**. Justificar siempre con evidencia del código ("el método hace X, Y y Z a la vez...", "estos 3 parámetros siempre viajan juntos...").

Smell (inglés)	Español	Cómo detectarlo	Refactoring principal
Duplicated Code	Código duplicado	Bloque idéntico o casi idéntico en 2+ lugares	Extract Method (+ Pull Up Method / Form Template Method)
Long Method	Método largo	Muchas líneas, variables temporales y ciclos que hacen difícil ver de un vistazo qué hace	Extract Method, Replace Temp with Query, Decompose Conditional
Large Class	Clase grande	Demasiados atributos/métodos/responsabilidades	Extract Class, Extract Subclass, Extract Interface
Long Parameter List	Lista larga de parámetros	Firmas con muchos parámetros (ej. 7) que se repiten en otros métodos	Introduce Parameter Object, Preserve Whole Object, Replace Parameter with Method
Data Clumps	Racimos de datos	Datos que siempre viajan juntos (fechaInicio , fechaFin)	Extract Class, Introduce Parameter Object, Preserve Whole Object
Feature Envy	Envidia de atributos/datos	Un método usa más datos de otra clase que de la propia	Move Method, Move Field, Extract Method
Data Class	Clase de datos	Solo atributos + getters/setters; la lógica vive en otras clases	Move Method, Encapsulate Field, Remove Setting Method

Smell (inglés)	Español	Cómo detectarlo	Refactoring principal
Switch Statement	Condicional por código de tipo	<code>switch/if-else if</code> sobre un tipo que crece con cada tipo nuevo	Replace Conditional with Polymorphism, Replace Type Code with Subclasses/State/Strategy
Primitive Obsession	Obsesión por primitivos	<code>String/int</code> para conceptos del dominio (moneda, tipo, CBU); comparación con <code>==</code> , sin validación	Replace Primitive with Object, Replace Type Code with Class, Introduce Parameter Object
Message Chains	Cadenas de mensajes	<code>a.getB().getC().getD().foo()</code>	Hide Delegate, Extract Method, Move Method
Middle Man	Intermediario	Clase/método que solo delega sin agregar valor	Remove Middle Man, Inline Class/Method, Replace Delegation with Inheritance
Inappropriate Intimacy	Intimidad inapropiada	Dos clases conocen detalles internos de la otra	Move Method/Field, Hide Delegate, Extract Class, Replace Inheritance with Delegation
Temporary Field	Campo temporal	Atributos que solo se usan en un método/rama; el resto del tiempo vacíos	Extract Class, Introduce Null Object
Refused Bequest	Herencia rechazada	La subclase no usa (o rechaza con excepciones) lo que hereda	Push Down Method/Field, Replace Inheritance with Delegation
Lazy Class	Clase perezosa	Clase que no hace nada que justifique su existencia (ej. <code>Avioneta</code>)	Inline Class, Collapse Hierarchy
Speculative Generality	Generalidad especulativa	Código "por si las moscas": métodos/parámetros/clases sin uso real	Remove Parameter, Collapse Hierarchy, Inline Class
Divergent Change	Cambio divergente	Una clase se modifica por motivos totalmente distintos según qué cambie	Extract Class
Shotgun Surgery	Cirugía de escopeta	Un cambio pequeño obliga a tocar muchas clases	Move Method/Field, Inline Class
Parallel Inheritance Hierarchies	Jerarquías paralelas	Cada subclase de una exige crear una de la otra	Move Method/Field

Smell (inglés)	Español	Cómo detectarlo	Refactoring principal
Comments	Comentarios excesivos	El comentario explica qué hace el código en vez de por qué	Rename Method, Extract Method, Replace Magic Number with Symbolic Constant
Alternative Classes with Different Interfaces	Clases alternativas con interfaces distintas	Dos clases hacen lo mismo con nombres/métodos distintos	Rename Method, Move Method, Extract Superclass
Incomplete Library Class	Clase de librería incompleta	La librería no tiene lo que necesitás y no podés modificarla	Introduce Foreign Method, Introduce Local Extension
Dead Store	Asignación muerta	Variable asignada y nunca leída (aparece en detección sobre AST)	Eliminar la asignación / Replace Temp with Query
Unused Parameter	Parámetro sin uso	Parámetro de la firma que nunca se referencia en el cuerpo	Remove Parameter
Redundant Conditional / Double Negation / Self Assignment	Condicionales redundantes	Ramas iguales (x ? 3 : 3), not not x , x = x , x and x	Simplificar / Decompose Conditional / Substitute Algorithm

Frases de justificación "tipo examen"

- **Long Method** → "El método hace A, B y C a la vez" → *Extract Method*.
- **Feature Envy** → "Se pasa la vida usando los datos de **OtraClase**" → *Move Method*.
- **Data Class** → "Solo guarda datos; la lógica vive en otras clases" → *Move Method*.
- **Duplicated Code** → "Si hay que corregirlo, hay que corregirlo en 3 lugares" → *Extract Method*.
- **Primitive Obsession** → "Usa un **String** para un concepto del dominio; pierde validación y **==** compara referencias" → *Replace Primitive with Object*.
- **Data Clumps** → "Estos 3 parámetros siempre viajan juntos" → *Introduce Parameter Object*.
- **Switch Statement** → "Cada tipo nuevo obliga a tocar este condicional" → *Replace Conditional with Polymorphism*.

3. Refactorings — los que más aparecen

Refactoring	Qué hace	Ejemplo
Rename Method / Variable	Nombre que diga qué hace (jes un refactoring de bajo costo y alto impacto!)	<code>lmtCrdt()</code> → <code>getLimiteCredito()</code>

Refactoring	Qué hace	Ejemplo
Extract Method	Saca un fragmento a un método con nombre propio	Fórmula del neto → <code>calcularNeto(e)</code>
Inline Method / Inline Class	Lo inverso: absorbe un método/clase trivial	<code>Catalogo</code> que solo delega
Move Method / Move Field	Mueve el miembro a la clase dueña de los datos	<code>Juego.incrementar()</code> → <code>Jugador.incrementarPuntuacion()</code>
Extract Class / Extract Superclass / Extract Subclass / Extract Interface	Nueva clase por cohesión, jerarquía o variación	6 atributos de dirección → <code>Direccion</code>
Pull Up / Push Down (Method/Field)	Subir lo común / bajar lo específico	Deducción común → clase padre <code>Empleado</code>
Encapsulate Field / Hide Method	Privado + acceso controlado	<code>public String nombre</code> → <code>private</code> + getter
Introduce Parameter Object / Preserve Whole Object	Reemplaza parámetros que viajan juntos por un objeto completo	<code>(fechaInicio, fechaFin)</code> → <code>(RangoFechas)</code>
Replace Temp with Query	Elimina variables intermedias consultando un método	<code>double p = calc(); fmt(p)</code> → <code>fmt(calc())</code>
Replace Primitive with Object	El primitivo pasa a ser objeto de dominio con validación	<code>"USD"</code> → <code>Moneda.USD</code>
Replace Conditional with Polymorphism	El <code>switch/if</code> de tipos se vuelve llamada polimórfica	<code>tipo.calcularImpuesto(monto)</code>
Replace Loop with Pipeline	Bucles manuales → Streams (filter/sorted/limit/sum)	Ordenar y tomar los N posts
Replace Magic Number with Symbolic Constant	Número mágico → constante con nombre	<code>0.13</code> → <code>PORCENTAJE_APORTES</code>
Form Template Method	Pasos comunes en un método concreto, variantes en primitivas	Notificador Email/SMS
Introduce Null Object	Sustituye <code>null</code> + chequeos por objeto con respuestas neutras	<code>FiltroVacio.aplicar(datos)</code>
Hide Delegate / Remove Middle Man	Ocultar una cadena de delegación / eliminar un delegado trivial	Inversos entre sí
Remove Setting Method / Remove	Eliminar setters/parámetros que no hacen falta	Estado que no debe mutar

Refactoring	Qué hace	Ejemplo
Parameter		
Substitute Algorithm	Cambiar el algoritmo por uno más simple	Selection sort → <code>sorted()</code>
Decompose Conditional	Extraer condición, ramas then/else a métodos con nombre	<code>if (x > 0 && ...)</code> complejo
Introduce Assertion	Documentar supuestos del código	Precondiciones
Change Bidirectional Association to Unidirectional	Reducir acoplamiento de asociaciones dobles	—

Orden típico de aplicación (regla de oro de los ejercicios):

1. *Extract Method* sobre lo duplicado (cambio seguro, un solo punto).
2. *Move Method* / *Encapsulate* para ordenar responsabilidades.
3. Recién ahí *Replace Conditional with Polymorphism* / *Replace Primitive with Object*.
4. **Iterar**: cada refactoring suele revelar otro smell ("Si vuelve a encontrar un mal olor, retorne al paso (i)").

4. Patrones de diseño

4.1 Template Method (comportamiento)

- **Propósito**: definir el **esqueleto de un algoritmo** en un método, difiriendo algunos pasos a las subclases. Las subclases redefinen pasos sin cambiar la estructura del algoritmo.
- **Roles**: `AbstractClass` (declara el `templateMethod()` — usualmente `final` — y las primitivas/hook), `ConcreteClass` (implementa las primitivas), `Client`.
- **Clave**: *primitiva* = paso abstracto obligatorio; *hook* = paso opcional con implementación por defecto (`hook()` devuelve `false`).
- **Cuándo**: cuando varias clases comparten el mismo algoritmo y solo cambian algunos pasos (Notificador Email/SMS, sueldos de Empleados).

4.2 Strategy (comportamiento)

- **Propósito**: definir una **familia de algoritmos**, encapsularlos e intercambiarlos en tiempo de ejecución.
- **Roles**: `Strategy` (interfaz común), `ConcreteStrategyA/B`, `Context` (guarda y cambia la estrategia), `Client`.
- **Clave**: el `Context` delega en la estrategia y puede cambiarla (`setStrategy`); el algoritmo varía independientemente del objeto que lo usa.

4.3 State (comportamiento)

- **Propósito**: que un objeto altere su comportamiento cuando su **estado interno** cambia ("parece cambiar de clase").

- **Roles:** **State** (interfaz del comportamiento por estado), **ConcreteStateA/B**, **Context** (guarda el estado actual).
- **Clave vs Strategy:** en **State**, cada estado **decide cuál es el próximo estado** (`context.setState(...)`), hay transiciones; en **Strategy**, el cliente cambia el algoritmo y no hay transiciones de estado.

4.4 Builder (creación)

- **Propósito:** separar la **construcción** de un objeto complejo de su **representación**, para que el mismo proceso construya representaciones distintas.
- **Roles:** **Director** (define el orden en `construct()`), **Builder** (interfaz de pasos `buildPartA/B/C` + `getResult`), **ConcreteBuilder**, **Product**.
- **Clave:** nuevo producto = nueva clase **ConcreteBuilder** (no se toca **Director**); **nuevo paso** de construcción = sí hay que tocar **Director** + **Builder** + concretas.

4.5 Factory Method (creación)

- **Propósito:** definir la interfaz para crear un objeto, dejando que las **subclases decidan qué clase instanciar**.
- **Roles:** **Creator** (declara `factoryMethod()` y usa el producto), **ConcreteCreatorA/B**, **Product**, **ConcreteProductA/B**.
- **Ejemplo:** `Logistica.crearTransporte()` → `LogisticaTerrestre` devuelve `Camion`, `LogisticaAerea` devuelve `Avion`.

4.6 Composite (estructural)

- **Propósito:** componer objetos en **estructuras de árbol** (parte-todo) y tratar **uniformemente** objetos individuales y composiciones.
- **Roles:** **Component**, **Leaf**, **Composite** (hijos: `add/remove/getChild`).
- **Clave:** solo se trata uniformemente si `add/remove/getChild` están en **Component** (la 2ª versión del UML del apunte); si solo están en **Composite**, no hay trato uniforme.

4.7 Decorator (estructural)

- **Propósito:** agregar **comportamiento adicional a un objeto en tiempo de ejecución** envolviéndolo, sin usar herencia explosiva.
- **Roles:** **Component**, **ConcreteComponent**, **Decorator** (implementa **Component** y **contiene** un **Component**), **ConcreteDecoratorA/B**.
- **Ejemplo:** `CafeSimple` decorado con `CafeConLeche` (describe + costo se acumulan). El **orden** de decoración importa.

4.8 Adapter (estructural)

- **Propósito:** hacer que **interfaces incompatibles** trabajen juntas: traduce la interfaz que el cliente espera a la que ofrece el **Adaptee**.
- **Roles:** **Target** (lo que espera el cliente), **Adaptee** (lo existente), **Adapter** (contiene el **Adaptee** y traduce `request()` → `specificRequest()`).

4.9 Proxy (estructural)

- **Propósito:** proveer un **sustituto/representante** que **controla el acceso** al objeto real (lazy init, permisos, cache, logging).
- **Roles:** **Subject**, **RealSubject**, **Proxy** (misma interfaz, delega con acciones antes/después).
- **Ejemplo:** **CamaraProxy** crea **CamaraReal** recién al primer uso y agrega mensajes.

4.10 Null Object (comportamiento — ¡aparece en la práctica!)

- **Propósito:** eliminar chequeos **null** con un objeto que implementa el protocolo completo con **respuestas neutras**.
- **Reglas:** **aplicar(datos)** → devuelve **datos** sin cambios; **esInvertido()** → **false**; consultas "sin valor" → centinela documentado (**Integer.MIN_VALUE** en el árbol vacío); los mutadores → no-op; los getters de hijos → devuelven **this** (el propio vacío).

Cuadro comparativo rápido

Pregunta en el parcial	Respuesta
Mismo algoritmo, pasos variables, herencia	Template Method
Algoritmos intercambiables, composición, el cliente elige	Strategy
Comportamiento según estado + transiciones entre estados	State
Construcción paso a paso de algo complejo	Builder
"¿Qué clase instanciar?" lo decide la subclase	Factory Method
Árbol parte-todo, trato uniforme	Composite
Envolver para agregar comportamiento en runtime	Decorator
Interfaces incompatibles	Adapter
Controlar el acceso al objeto real	Proxy
Sacar if (x != null) de todos lados	Null Object

5. Detección de smells sobre el AST (ANTLR / Pool)

Técnica general: **recorrer el árbol** identificando tipos de nodo (función, asignación, condicional, llamada, acceso a atributo) y aplicando un patrón de detección por smell. Siempre: *recorrer* → *detectar* *patrón* → *reportar* (a veces con un set/contador y un umbral).

Smell detectable	Pseudocódigo esquema
Dead Store (variable asignada y nunca leída)	Llevar un set de variables. Por cada referencia: si está a la izquierda de una asignación → agregarla; si está a la derecha (se usa) → quitarla. Al final, las que quedan en el set → reportar.
Unused Parameter	Inicializar setParámetros con la firma. Por cada uso de parámetro en el cuerpo → eliminarlo del set. Al final, los que quedan → reportar.

Smell detectable	Pseudocódigo esquema
Redundant Conditional (<code>x ? 3 : 3</code>)	Por cada condicional: extraer subárbol de rama verdadera y de rama falsa; si son estructuralmente iguales → reportar.
Double Negation (<code>not not y</code>)	Por cada nodo not : si su hijo es otro not → reportar.
Redundant Boolean (<code>x and x</code> , <code>x or x</code>)	Por cada operador lógico: comparar subárbol izquierdo y derecho; si son idénticos → reportar.
Tautología / Contradicción (<code>x or not x</code>)	Igual que arriba, pero si un subárbol es la negación exacta del otro → reportar.
Self Assignment (<code>x = x</code>)	Por cada asignación: si subárbol izquierdo y derecho son idénticos → reportar.
Contradictory Conditions / Simulated Else (<code>x ? ...; not x ? ...</code>)	Por bloque de sentencias, registrar condiciones vistas; si aparece not <condición ya vista> (o viceversa) → reportar.
Duplicate Function Call (<code>g.x()</code> + <code>g.x()</code>)	Contador por (nombre de función + argumentos); si se llama 2+ veces con los mismos argumentos → reportar (guardar en variable).
Duplicated Code (11 llamadas idénticas)	Registrar subárboles de llamadas excluyendo hojas/argumentos ; si un subárbol se repite más de un umbral → reportar (o reemplazar por bucle).
Feature Envy / Envidia de atributos	Contador de accesos a atributos por objeto ; si this aparece menos que otro objeto (contador de otro objeto > 1) → reportar.
Switch Statement	Detectar comparaciones de tipo (<code>instanceof</code> , <code>.class</code> , <code>typeof</code> , <code>.equals</code> sobre "variables de tipo") repetidas; contador > 1 → reportar.
Long Parameter List	Contar nodos de parámetro en la firma; si supera un umbral (4-5) → reportar.
Middle Man puro	Función cuyo cuerpo es exactamente una llamada cuyos argumentos son referencias directas a los parámetros (mismo orden, sin operaciones) → reportar.

6. Lo esencial de la práctica de cursada (ResolucionPractiaCursada)

- Protocolo de Cliente** (`lmtCrdt`, `mtFcE...`) → nombres poco descriptivos (**Rename Method**), parámetros `f1/f2` (**Rename Variable** + **Introduce Parameter Object** → `RangoFechas`), visibilidad a revisar.
- Participación en proyectos** → mover `participaEnProyecto` de `Persona` a `Proyecto` (**Move Method**); `id` público → **Encapsulate Field**.
- Cálculo de promedios/salarios** → **Extract Method** + **Replace Temp with Query** + streams ("reinventando la rueda" → *Replace Loop with Pipeline*).
- Empleados (Temporario/Planta/Pasante)** → iteración clásica: (1) Duplicated Code → **Extract Method** `calcularDeducion()`; (2) atributos repetidos → **Extract Superclass** `Empleado`; (3) esqueleto común → **Template Method**; (4) atributos públicos → **Encapsulate Field**; (5) método largo → **Extract Method**.

5. **Juego/Jugador** → atributos públicos (**Encapsulate Field**) + **Feature Envy** → **Move Method** (`incrementarPuntuacion` pasa a `Jugador`).
6. **Publicaciones (ultimosPosts)** → **Long Method** → **Extract Method** (filtrar / ordenar / tomar N) y luego **Replace Loop with Pipeline** con streams. (Ojo el bug del `index++` faltante: arreglarlo **no** es refactoring.)
7. **Carrito de compras** → **Feature Envy**: `total()` conoce datos de `Producto/ItemCarrito` → **Move Method** (`ItemCarrito.calcularTotal()`) + **Rename Method**.
8. **Envío de pedidos** → `Cliente.getDireccionFormateada()` es **Feature Envy** → **Move Method** a `Direccion`; **Encapsulate Field**; posible **Middle Man** → pasar `Direccion` directo.
9. **Películas (HBOO)** → `switch` por tipo de suscripción → **Replace Conditional with Polymorphism** (`TipoSuscripcion`), **Feature Envy** → **Move Method**, comentario innecesario → **Extract Method**.
10. **Document (characterCount/calculateAvg)** → Duplicated Code → **Extract Method** `calcularSumaLongitudes()`. ⚠ **Pregunta clave del parcial**: el promedio devuelve `long` (pierde decimales) y hay división por cero si `words` está vacía. Arreglarlo **cambia el comportamiento** (firma/excepciones), por lo tanto **NO es refactoring**.
11. **Pedidos (Pedido.getCostoTotal)** → secuencia pedida: **Replace Loop with Pipeline** (suma de precios), **Replace Conditional with Polymorphism** (`FormaPago` → `Efectivo/SeisCuotas/DoceCuotas`), **Extract Method** + **Move Method** (`Cliente.getAniosDesdeFechaAlta()`), **Extract Method** + **Replace Temp with Query** (`aplicarDescuentoSiCorresponde`). Notar que el constructor validador de `formaPago` desaparece porque el tipo ahora es un objeto.
12. **Facturación de llamadas (Empresa/Cliente/Llamada)** → recorrido largo: **Move Method** (`agregarNumeroTelefono` → `GestorNumerosDisponibles`), **Replace Temp with Query** (variable `encontre`), **Extract Method** (`crearCliente`), encapsular `llamadas` (**Hide Method** `agregarLlamada`), **Replace Conditional with Polymorphism** en dos frentes (`Cliente/ClienteFisica/ClienteJuridica` con `aplicarDescuento`, y `Llamada/LlamadaNacional/LlamadaInternacional` con `calcularCosto`), **Rename Variable** (`c`, `auxc`), y el `switch` de `obtenerNumeroLibre` → interfaz `TipoGenerador` (¡es Strategy!).
13. **Árbol binario** → Duplicated Code + chequeos `null` en los 3 recorridos → **Introduce Null Object**: interfaz `IArbolBinario` + `ArbolBinarioVacio` (recorridos devuelven "", `getValor()` devuelve `Integer.MIN_VALUE`, hijos devuelven `this`). Los recorridos quedan en una línea sin `if != null`.

7. Lo esencial de la práctica "Deep into Refactoring" (detección sobre AST)

Ejercicios con el lenguaje de la cátedra (el valor de retorno de una función es el de la **última sentencia**):

Caso	Código típico	Smell	Detección
A / B / D	<code>a = 4; ...</code> sin leer <code>a</code>	Dead Store (+ Unused Parameter en B/D)	set de variables/parámetros
C / H	parámetro sin usar	Unused Parameter	set de parámetros
E	<code>a = x ? 3 : 3</code>	Redundant Conditional	comparar ramas
F	<code>g.x() + g.x() + g.x()</code>	Duplicate Function Call	contador por función+argumentos

Caso	Código típico	Smell	Detección
G	<code>not not y</code>	Double Negation	hijo <code>not</code> de un <code>not</code>
I	<code>clase.equals(pasante) ? ...</code>	Switch Statement	comparaciones de tipo repetidas
J	<code>11 lista.agregar(n)</code>	Duplicated Code	subárboles repetidos (sin hojas)
K	<code>telefono.codigoArea + ...</code>	Feature Envy	contador de accesos por objeto
L	<code>x or not x, x and x, x ? x ? ...</code>	Redundant Conditional / Tautología / condición anidada redundante	comparar subárboles y ancestros
M	<code>x = x</code>	Self Assignment	izquierda == derecha
N	<code>x ? ...; not x ? ...</code>	Contradictory Conditions (Simulated Else)	condiciones registradas vs su negación
Ñ	<code>f(a,...,k)</code>	Long Parameter List	contar parámetros > umbral
O	<code>other.someOperation(x,y,z)</code>	Middle Man	cuerpo = 1 llamada con los mismos parámetros

Repaso.md (árboles): (1) condicional con ramas iguales → *Conditionals Duplicated*; (2) parámetro `x` no usado → *Unused Parameter*. Mismo esquema de recorrido + reporte.

8. Práctica "Bad Smells" — mapa y respuestas rápidas

Mapa: Ej 1 nómina (Duplicated Code, Long Method, Comments) · Ej 2 reservas hotel (Switch, Primitive Obsession, Data Clumps, Long Parameter List) · Ej 3 facturas (Data Class, Feature Envy, Message Chains) · Ej 4 vehículos (Refused Bequest, Lazy Class, Speculative Generality) · Ej 5 biblioteca (Middle Man, Temporary Field, Inappropriate Intimacy) · Ej 6 PHP (Duplicated Code, Switch, Primitive Obsession, Long Method) · Ej 7 impuestos (Replace Conditional with Polymorphism) · Ej 8 Persona (Extract Class + Introduce Parameter Object) · Ej 9 transferencias (Replace Primitive with Object) · Ej 10 reportes (Introduce Null Object) · Ej 11 notificaciones (Form Template Method) · Ej 12 cadena de delegación (Hide Delegate) · Ej 13 opción múltiple · Ej 14 V/F.

Opción múltiple (respuestas):

Ítem	Respuesta
13.1 método de 90 líneas con variables temporales y ciclos	B) Long Method
13.2 <code>fechaInicio</code> , <code>fechaFin</code> , <code>horaCheckIn</code> viajan juntos	C) Data Clumps

Ítem	Respuesta
13.3 definición de Feature Envy	A) método que usa principalmente datos de otra clase
13.4 switch que crece con cada tipo	B) Replace Conditional with Polymorphism
13.5 método que solo llama a otro.metodo()	B) Remove Middle Man
13.6 Vendedor con solo getters/setters	B) Data Class → Move Method

Verdadero/Falso (justificación corta):

- **14.1 FALSO** — el refactoring **no** puede cambiar el comportamiento observable.
- **14.2 VERDADERO** — *Extract Method* ataca duplicación y longitud (para jerarquías: + *Pull Up / Form Template Method*).
- **14.3 VERDADERO** — *Large Class* → *Extract Class* (o *Extract Subclass/Interface*).
- **14.4 FALSO (matiz)** — *Replace Primitive with Object* y *Replace Conditional with Polymorphism* son distintos pero **complementarios y encadenados**.
- **14.5 VERDADERO** — el Null Object responde de forma neutra y elimina chequeos **null**.
- **14.6 FALSO** — *Rename Method* es de alto impacto: los nombres son la documentación primaria (un nombre críptico es un smell en sí mismo).

Detalle técnico del Ej 2 (hotel): `tipoHabitacion == "simple"` compara **referencias** en Java (bug latente) — el *Replace Primitive with Object* lo elimina de raíz.

9. Checklist mental para el parcial

1. **Detectar** → nombrar el smell **en inglés** y citar evidencia concreta (qué método, qué datos usa, qué se repite).
2. **Elegir el refactoring que ataca la CAUSA**, no el síntoma: un **switch** largo no se arregla con *Extract Method* solo.
3. **Aplicar en pasos pequeños**, verificando que el comportamiento no cambió.
4. **Iterar**: cada refactoring revela otro smell.
5. Recordar el límite: si hay que cambiar el **comportamiento** (firma, bug, resultado), eso **no es refactoring**.
6. Cuando pidan "aplicar X", mostrar **firmas y clases resultantes** (y UML si lo piden).
7. Ante un diseño con tipos en **String** + **if/else** → *Replace Primitive with Object* + *Replace Conditional with Polymorphism*.
8. Ante **a.getB().getC().foo()** → *Hide Delegate*; ante una clase que solo delega → *Remove Middle Man*.
9. Ante **if (x != null)** repetidos → *Introduce Null Object* (respuesta neutra bien elegida).
10. Ante algoritmo común + pasos variables en subclases → *Form Template Method* / **Template Method**.